

# Advancs System Analysis

**Dr. Nesma Ebrahim**

**Lecture 5**

**Reuse in Sytems—Architectural Patterns**

# LECTURE OBJECTIVES:

- Introduce the concept of **Reuse in Software Engineering**.
- Introduce the concepts of **Architectural Diagrams & Patterns**.
- Discuss why architectural patterns are important and **what advantages** they provide.
- Discuss 10 selected architectural patterns.

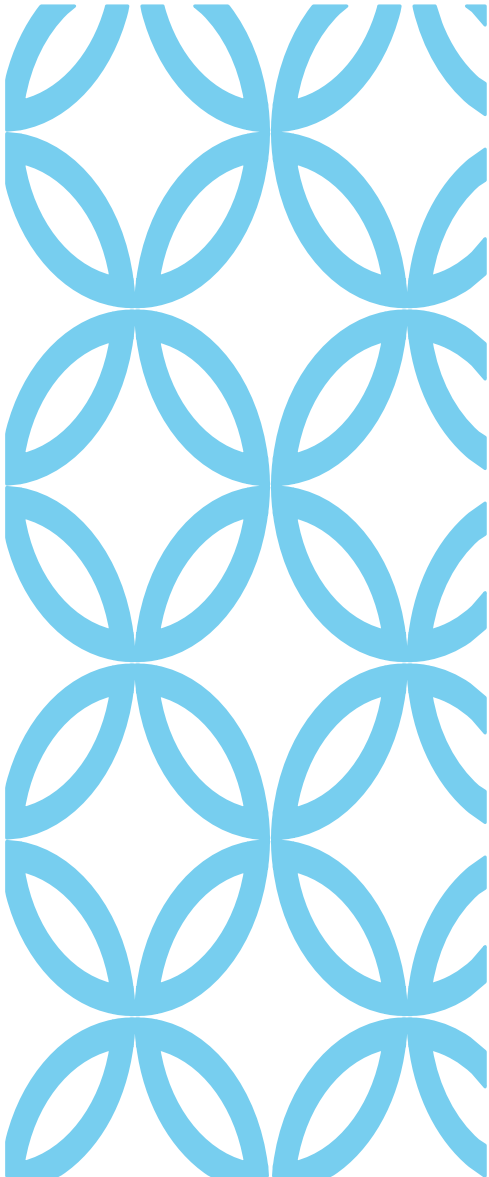


# REUSE IN SOFTWARE ENGINEERING

A major aspiration of software engineering is reuse; *taking what you or others have done or created in the past and using it either unchanged or with relatively little adaptation.*

Following are some reasons why reuse is desirable:

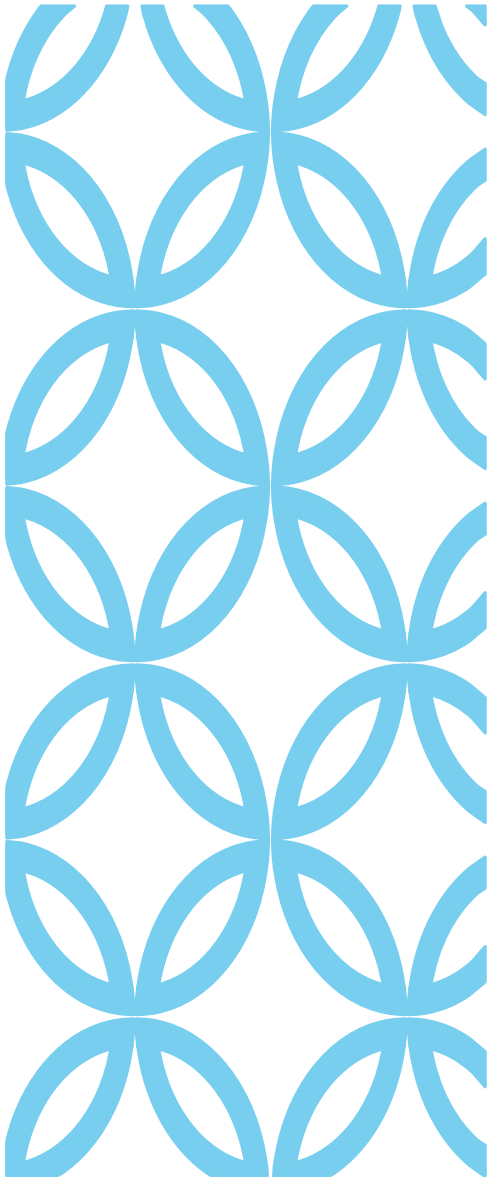
- It **avoids duplication of effort** & reinventing already existing solutions, which saves resources.
- It **promotes reliability** because developers can use tried & trusted solutions.
- It **speeds up development** because developers can take existing solutions without starting from the beginning every time.
- It is a mechanism for **spreading good practice** among the software development community & familiarizing practitioners (developers) with tried & tested solutions.



- A **pattern** is the outline of a reusable solution to a recurring problem encountered in a particular context.
- Many of them have been systematically documented for all software developers to use.
- A good pattern should contain a solution that has been proven to effectively solve the problem in the indicated context.
- ***Studying patterns is an effective way to learn from the experience of others***

---

## INTRODUCTION TO PATTERNS

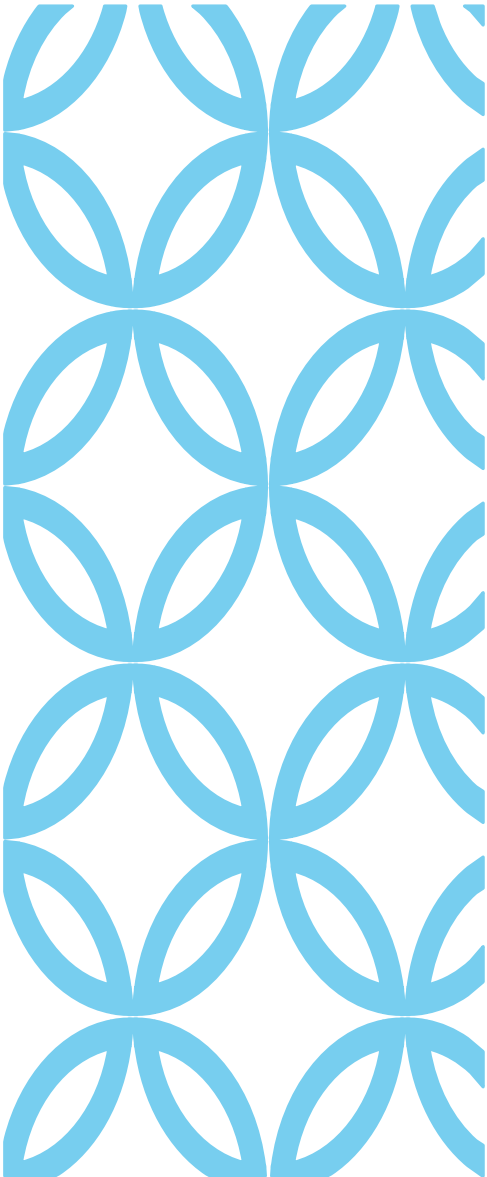


Patterns provide **common language** between developers. For example if a developer tells another developer that he is using a ***Singleton***, the other developer should know exactly what this means.

They help to **improve software quality & reduce development time**. As They provide proven solution and are based on the basic principles and heuristics of object oriented design.

---

## MOTIVATION FOR PATTERNS



Reuse exists on several levels. Thus, Software Patterns include the following types:

- 1) Requirements Patterns
- 2) Analysis Patterns
- 3) **Architectural Patterns**
- 4) Assigning responsibilities patterns
- 5) **Design Patterns**
- 6) Idioms

---

## SOFTWARE PATTERNS

SOFTWARE  
ARCHITECTURAL  
DESIGN &  

---

ARCHITECTURAL  
PATTERNS/STYLES



# SOFTWARE ARCHITECTURAL DESIGN

## DEFINITIONS

*“The software architecture of a program or computing system is the structure or structures of the system, which comprise software elements, the externally visible properties of those elements, and the relationships among them.”*

A **software architecture** separates the overall structure of the system, in terms of components and their interconnections, from the internal details of the individual components.

The emphasis on components and their interconnections is sometimes referred to as **programming-in-the-large**, and the detailed design of individual components is referred to as **programming-in-the-small**.

A software architecture can be described at **different levels of detail**. At a high level, it can describe the decomposition of the software system into subsystems. At a lower level, it can describe the decomposition of subsystems into modules or components. In each case, the emphasis is on the external view of the subsystem/component – that is, the interfaces it provides and requires – and its interconnections with other subsystems/components.

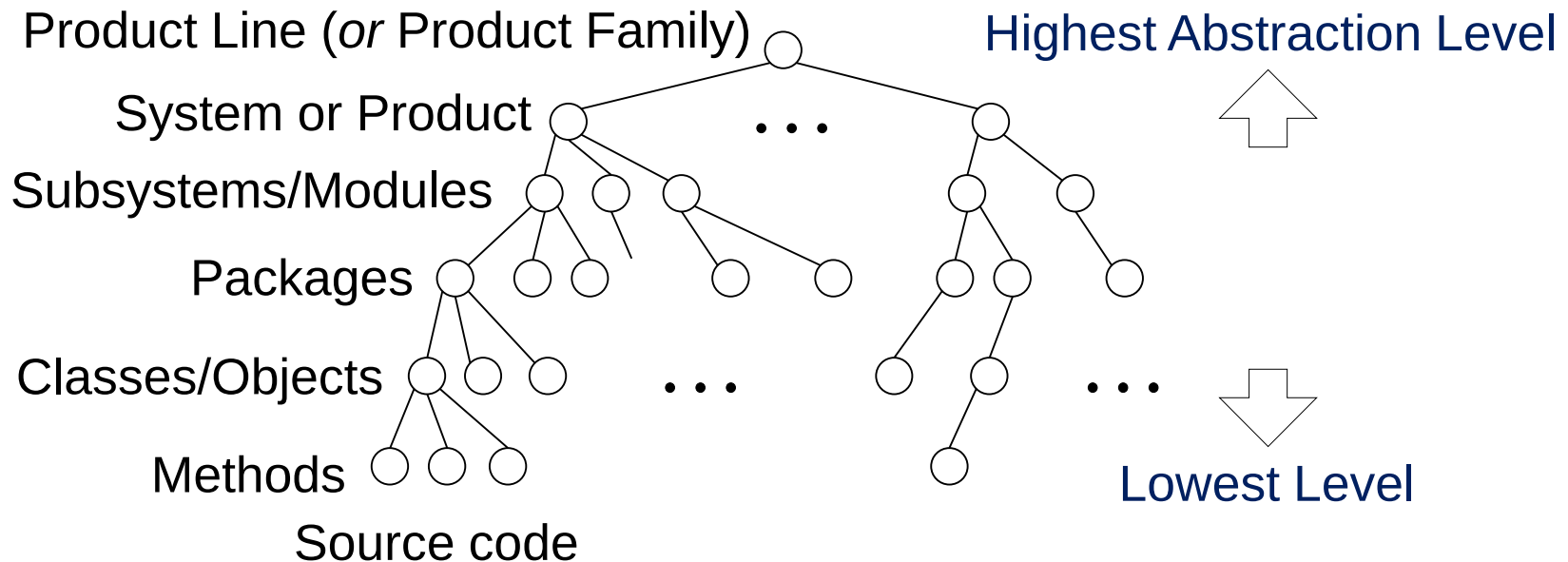
# SOFTWARE ARCHITECTURAL DESIGN ..

## WHY DECOMPOSE SYSTEMS?

- Tackle complexity by “divide-and-conquer”.
- See if some parts already exist & can be reused.
- Support flexibility and future evolution by decoupling unrelated parts, so each can evolve separately (“separation of concerns”).

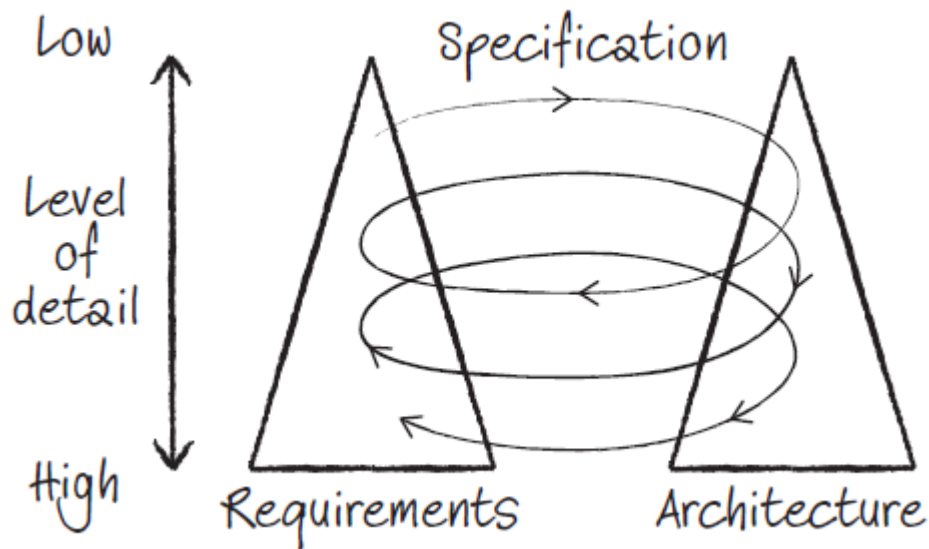
# SOFTWARE ARCHITECTURAL DESIGN ..

## DIFFERENT LEVELS OF DETAIL



# SOFTWARE ARCHITECTURAL DESIGN .. & .. REQUIREMENTS

It is important to ensure that the architecture **fulfills** the software **requirements**, both **functional** (what the software has to do) and **nonfunctional** (how well it should do it).



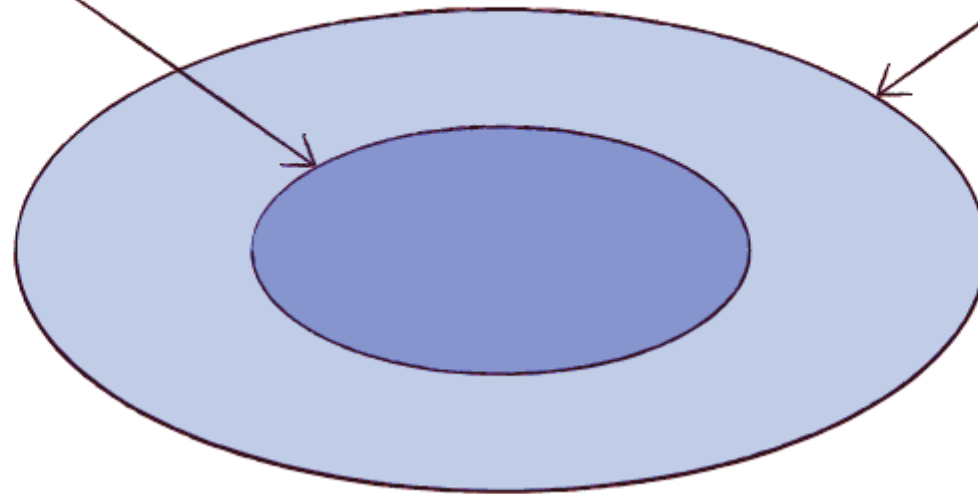
◀ Twin-peaks model (Nuseibeh, 2001), which develops requirements & architecture concurrently and iteratively.

# SOFTWARE ARCHITECTURAL DESIGN .. & .. FUNCTIONAL VS. NON-FUNCTIONAL REQ.

There can be multiple architectures that meet functional requirements but they **will not all be equal when it comes to non-functional requirements**. Moreover, not all non-functional requirements will be of equal importance when making architectural decisions.

Architectures that also meet non-functional requirements

Architectures that meet functional requirements



# SOFTWARE ARCHITECTURAL DESIGN .. & .. NON-FUNCTIONAL REQUIREMENTS

The software quality attributes of a system should be considered when developing the software architecture. These attributes relate to how the architecture **addresses important nonfunctional requirements**, such as performance, security, and maintainability.

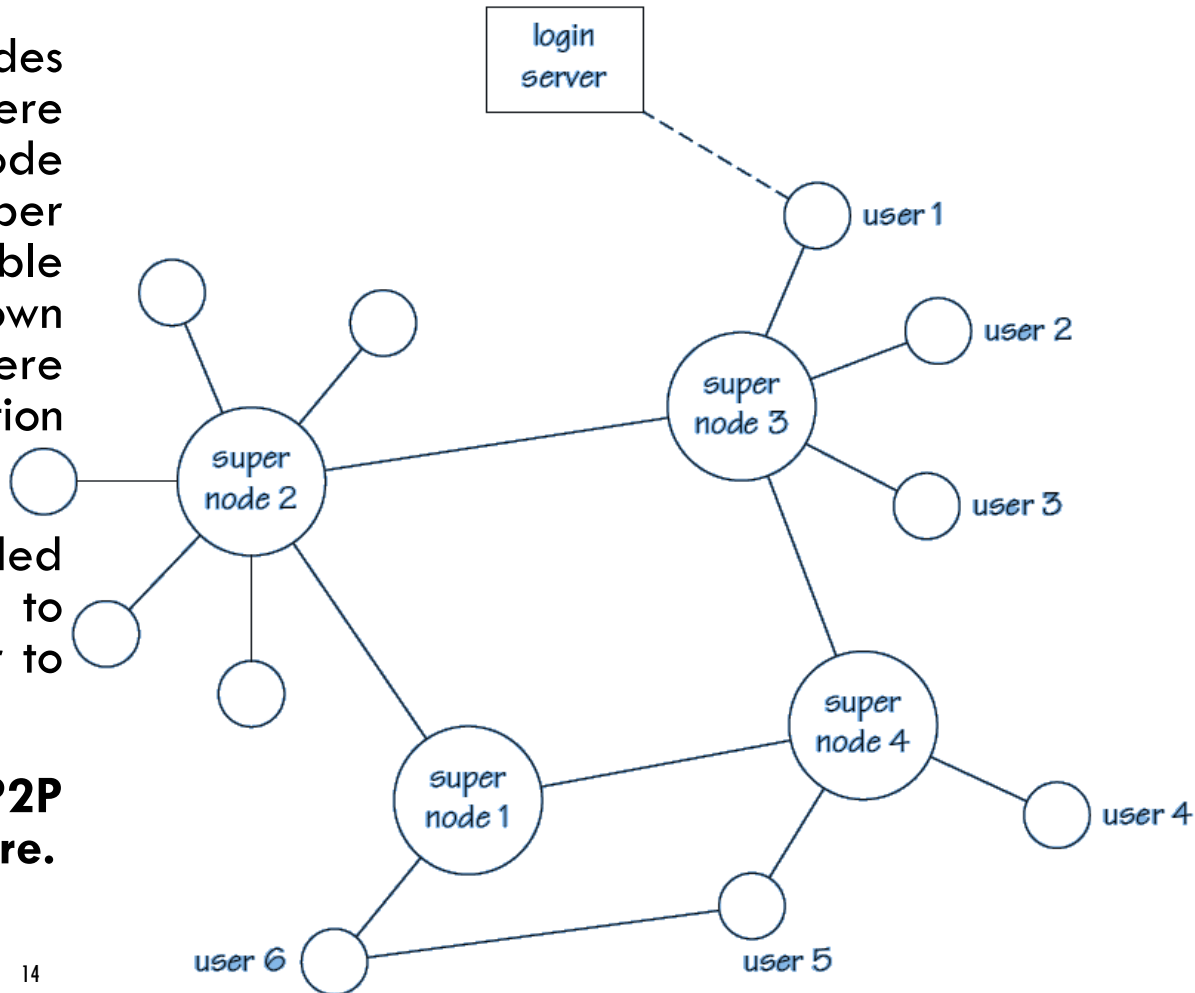
**Example:** How **Skype's architecture** is affected by a non-functional requirement (i.e., **Reliability**)

- Skype is a well known internet telephony service.
- To go online, and individual use must first contact a login server, and it then connected to the nearest super node.
- The super-nodes then route telephone calls from one user to another, yet it's also possible for two users to be connected directly.

# SOFTWARE ARCHITECTURAL DESIGN .. & ..

## NON-FUNCTIONAL REQUIREMENTS

- **Originally**, all the nodes except the login server were equivalent. Any suitable node could be promoted to a super node and become responsible for routing calls (a style known as **peer-to-peer "P2P"** where there is no clear distinction between clients and servers).
- **Subsequently**, it was decided to move the super nodes to Skype's own servers in order to improve reliability.
- Skype now has **a mixed P2P and Client-Server Architecture**.



# SOFTWARE ARCHITECTURAL DESIGN

## ARCHITECTURAL PATTERNS/STYLES

Software architectural patterns provide the **skeleton or template for the overall software architecture** or high-level design of an application.

**Architectural styles or patterns of software architecture:** *are recurring architectures used in a variety of software applications.* These include such widely used architectures as client/server and layered architectures.

Software architectural patterns can be grouped into two main categories: (1) **architectural structure patterns**, which address the static structure of the architecture, and (2) **architectural communication patterns**, which address the dynamic communication among distributed components of the architecture.

# SOFTWARE ARCHITECTURAL DESIGN

## ARCHITECTURAL PATTERNS/STYLES

Commonly employed architectural styles include:

- Client–Server
- Call-Return
- Layered
- Peer-to-Peer
- Data-flow
- Data-Centred
- Independent Components
- Service-Oriented
- Notification
- Model-View-Controller

Important Note: there are relationships between these different styles and some overlap with others.

# SOFTWARE ARCHITECTURAL STYLES

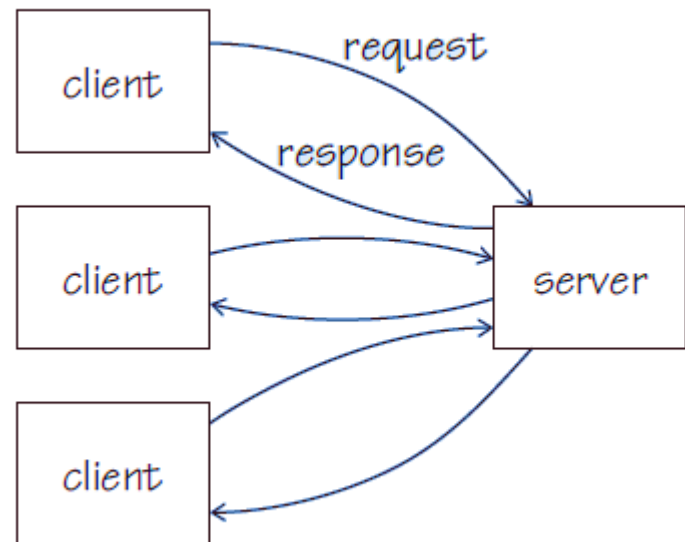
## CLIENT-SERVER ARCHITECTURE

Probably the best known of all architectural styles.

One component (the Server) provides a service to the other components (the Clients).

A server waits for requests from clients, processes them when received, and returns a response to the client.

**Examples:** requests from web-browsers to web-servers.

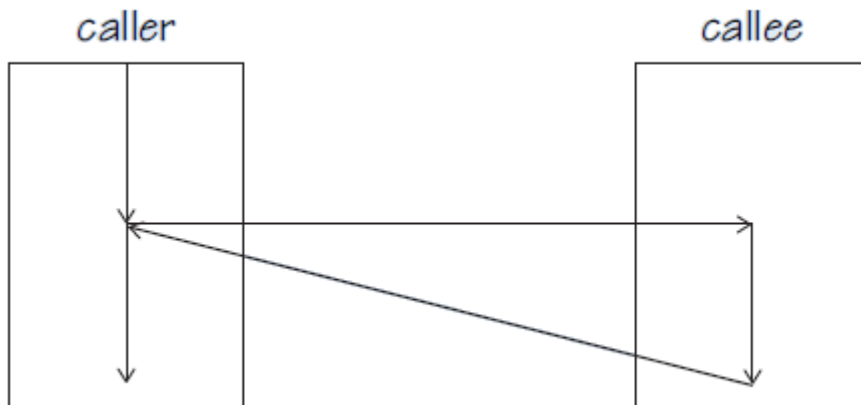


# SOFTWARE ARCHITECTURAL STYLES

## CALL-RETURN ARCHITECTURE

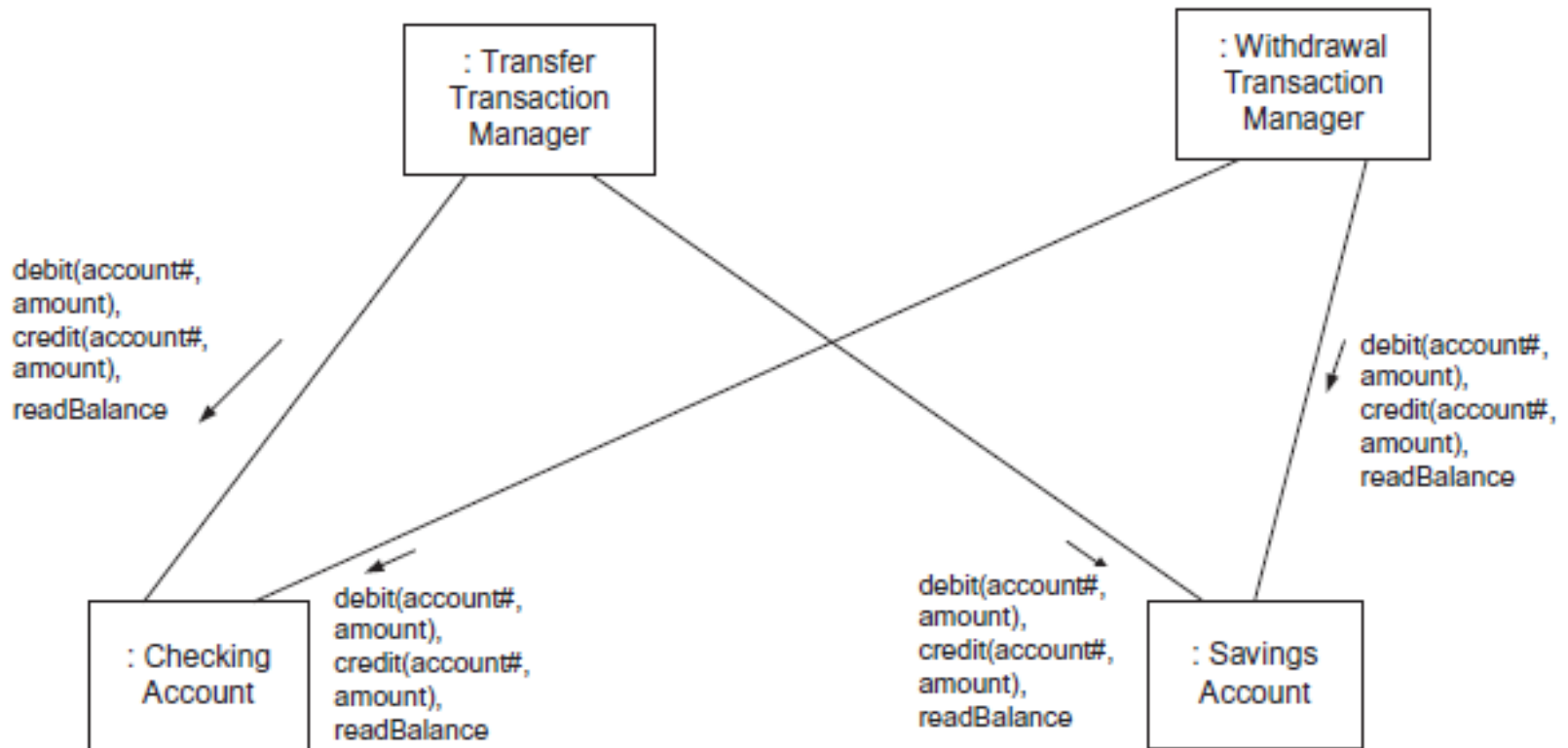
A component (the Caller) makes a procedure call to another component (the Callee) and waits for the call to return.

**Examples:** Main program calls to a subprogram (in a traditional/structural software) or methods invocation (in OOP).



# SOFTWARE ARCHITECTURAL STYLES

## CALL-RETURN ARCHITECTURE



# SOFTWARE ARCHITECTURAL STYLES

## LAYERED (TIERED) ARCHITECTURE

The system is structured as a series of layers that are visualized as stacked on top of another.

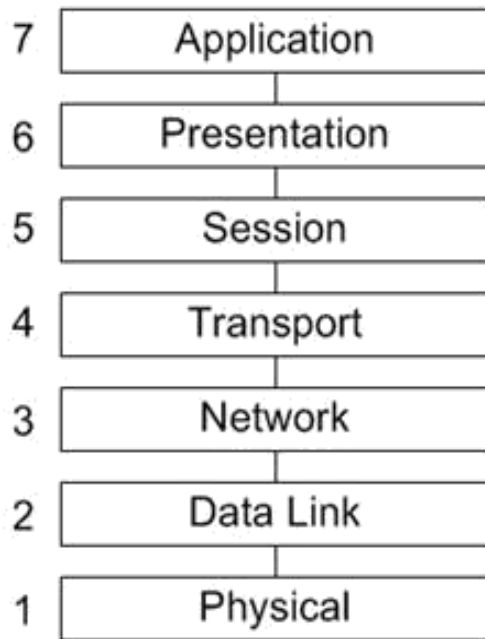
Each layer uses services provided by the layers below (usually just by the layer immediately below), and it supplies services to the layer above.

### Examples:

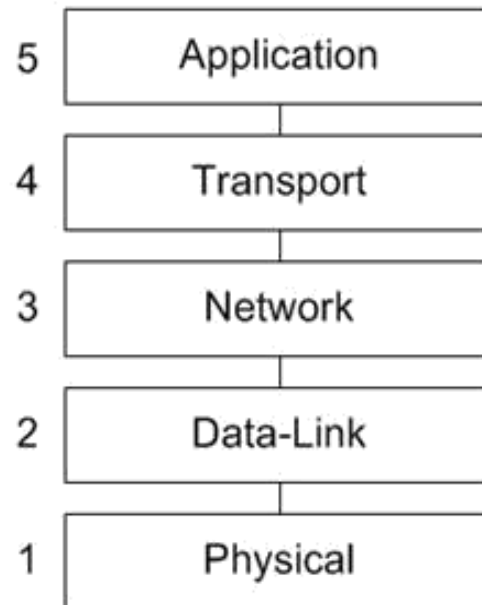
- A compiled Java program, which executes in a Java Virtual Machine that in turn makes calls to services supplied by the Operating system.
- Open Systems Interconnection (OSI) & Transmission Control Protocol/Internet Protocol (TCP/IP).
- Client-Server Architecture is a special case in which there are just 2 layers.

# SOFTWARE ARCHITECTURAL STYLES

## LAYERED (TIERED) ARCHITECTURE



OSI Model



TCP/IP Model

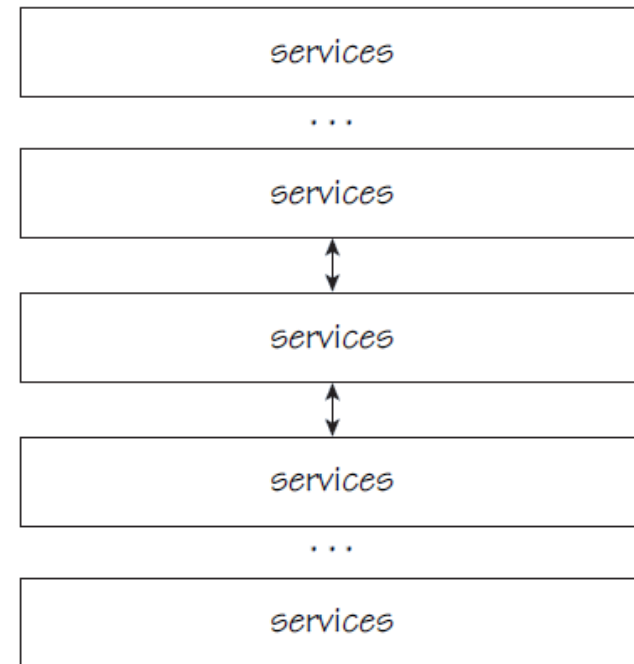
top layer

layer n+1

layer n

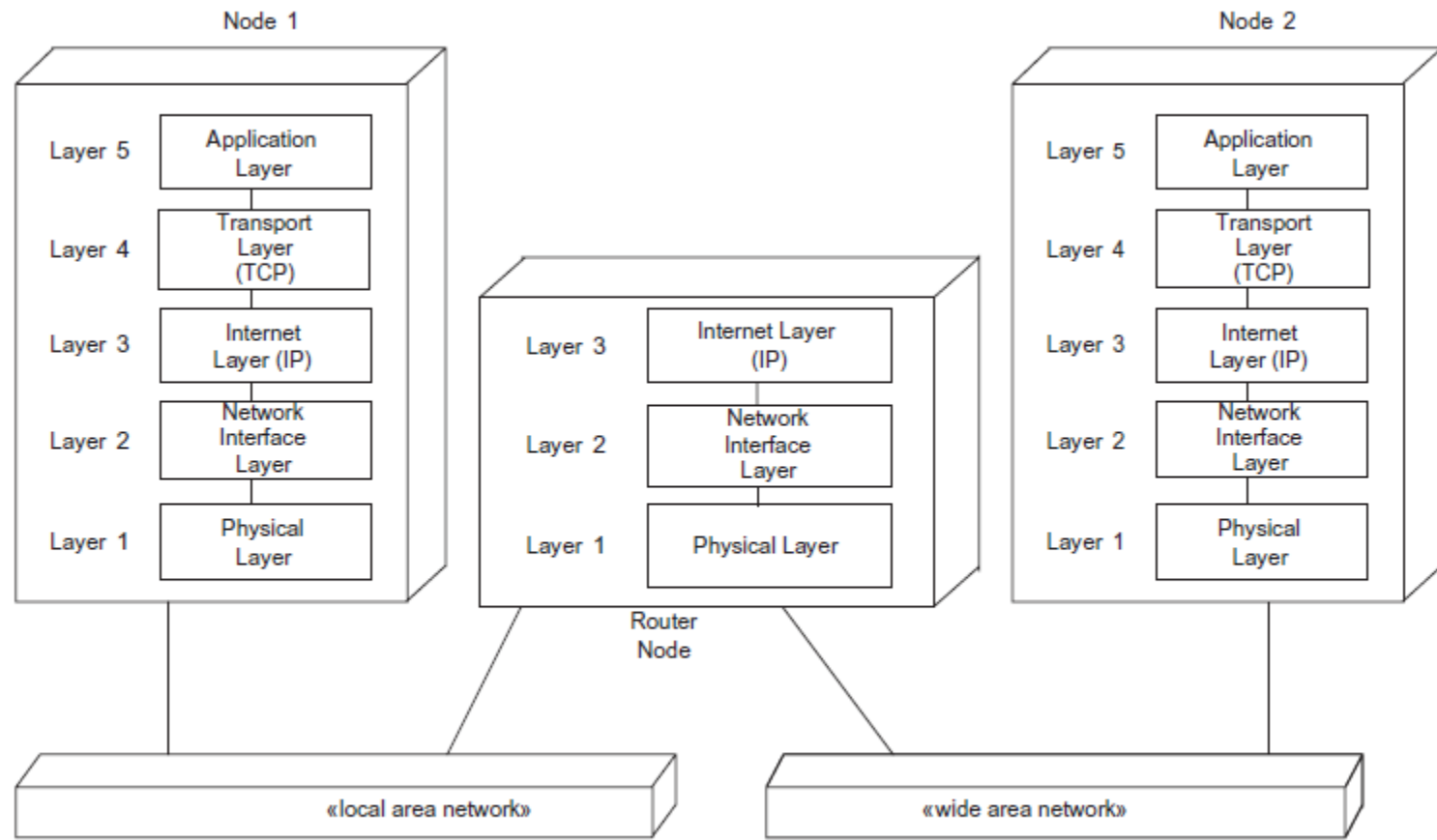
layer n+1

bottom layer



# SOFTWARE ARCHITECTURAL STYLES

## LAYERED (TIERED) ARCHITECTURE



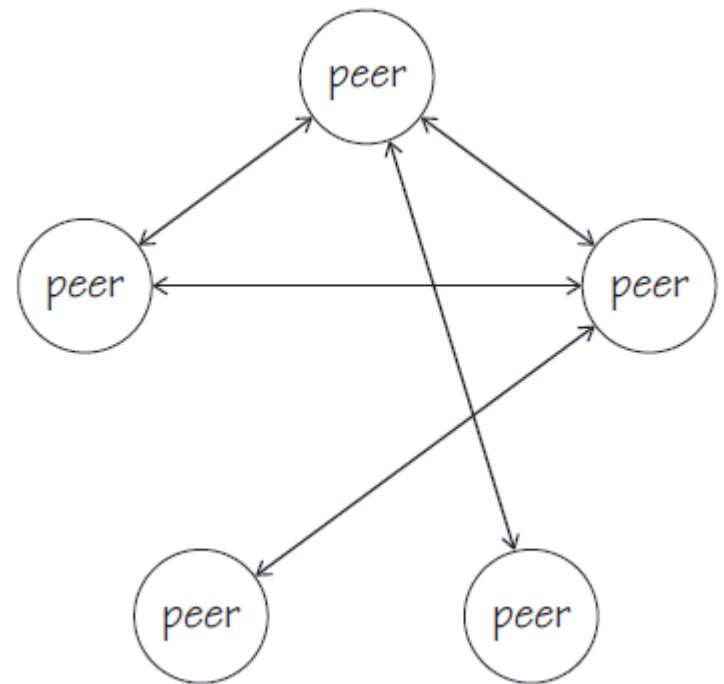
Layers of Abstraction architectural pattern: Internet communication with TCP/IP

# SOFTWARE ARCHITECTURAL STYLES

## PEER-TO-PEER ARCHITECTURE

Peer-to-Peer resembles the Client-Server style except that all the components are both clients and servers and any component can request services from any other component.

**Example:** a streaming music service, where listeners generally get streamed tracks from the nearest peer than can be located by sending a request that hops from one peer to another until the desired track is located.



# SOFTWARE ARCHITECTURAL STYLES

## DATA-FLOW (PIPES & FILTERS) ARCHITECTURE

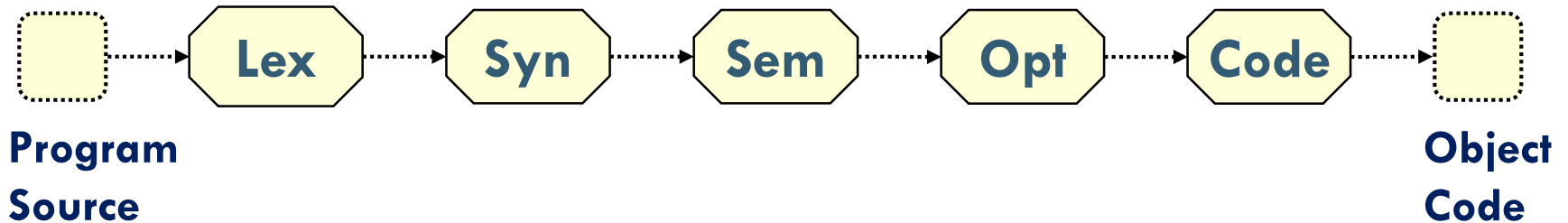
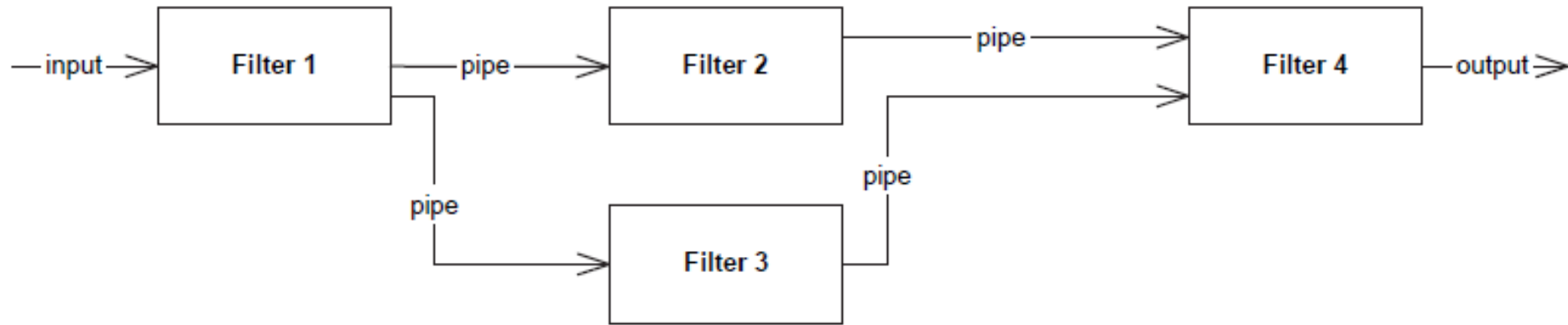
The data-flow style components are objects or small independent subprograms (filters) that process a stream of data and pass the results on to other components for further processing.

Communication is unidirectional and uses fixed channels, and each filter has no knowledge of other filters upstream or downstream (simply accepts data, processes it, and then passes it on).

**Example:** A Pipelined Compiler Architecture.

# SOFTWARE ARCHITECTURAL STYLES

## DATA-FLOW (PIPES & FILTERS) ARCHITECTURE



# SOFTWARE ARCHITECTURAL STYLES

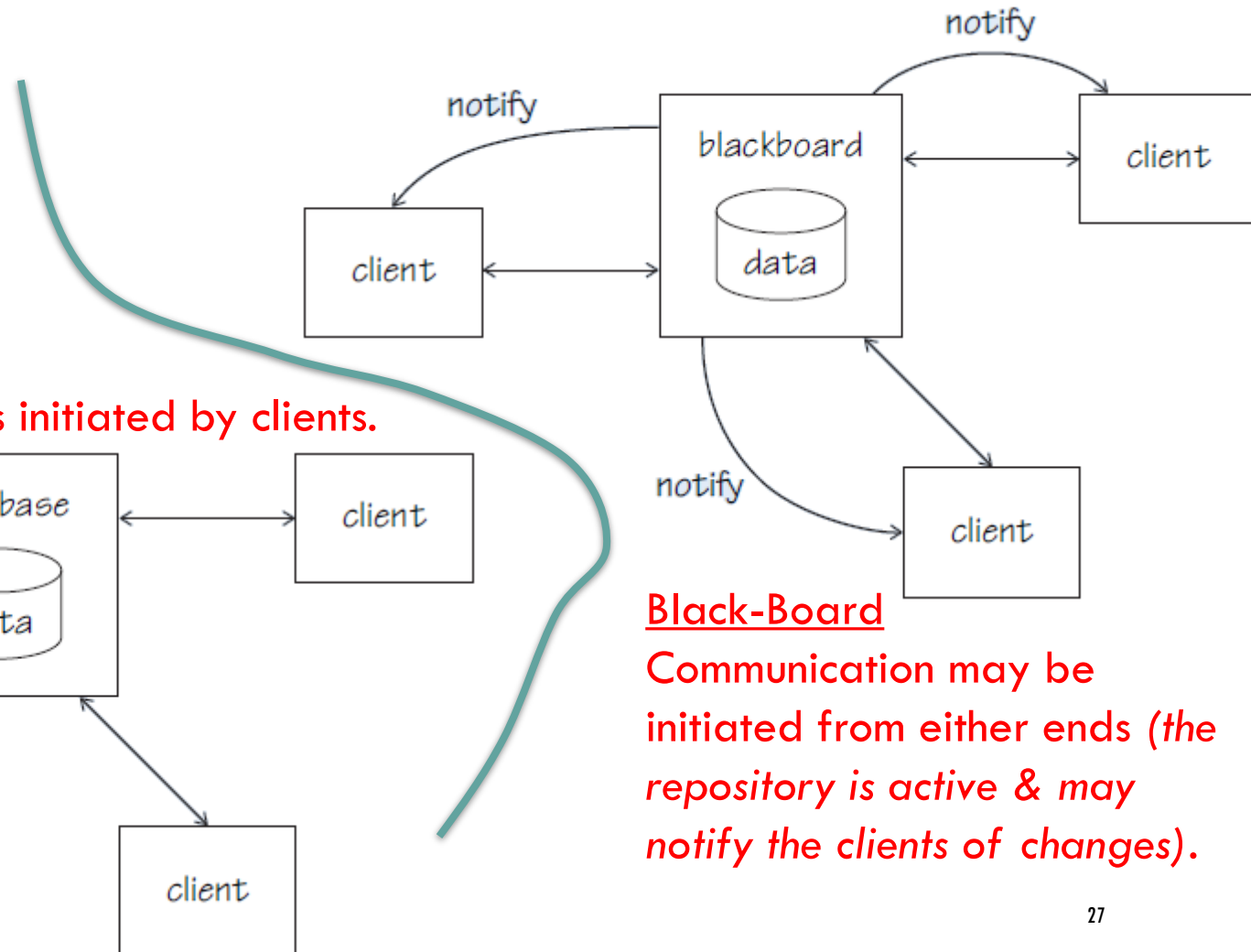
## DATA-CENTRED (REPOSITORY / BLACK-BOARD)

- There is a data provider that is a centralized store of persistent data.
- The structure of the data, the types of items & their relationships are stable (change rarely or not at all).
- Many Clients (who are data consumers) can have items created, queried, updated, & destroyed upon request.
- The central store may be duplicated to provide backup (in case of failure) or deal with a greater volume of requests.

**Example:** A database holding personnel records with authorized users being able to log on and submit queries.

# SOFTWARE ARCHITECTURAL STYLES

## DATA-CENTRED (REPOSITORY / BLACK-BOARD)



### Database (Repository)

Communication is always initiated by clients.

### Black-Board

Communication may be initiated from either ends (the repository is active & may notify the clients of changes).

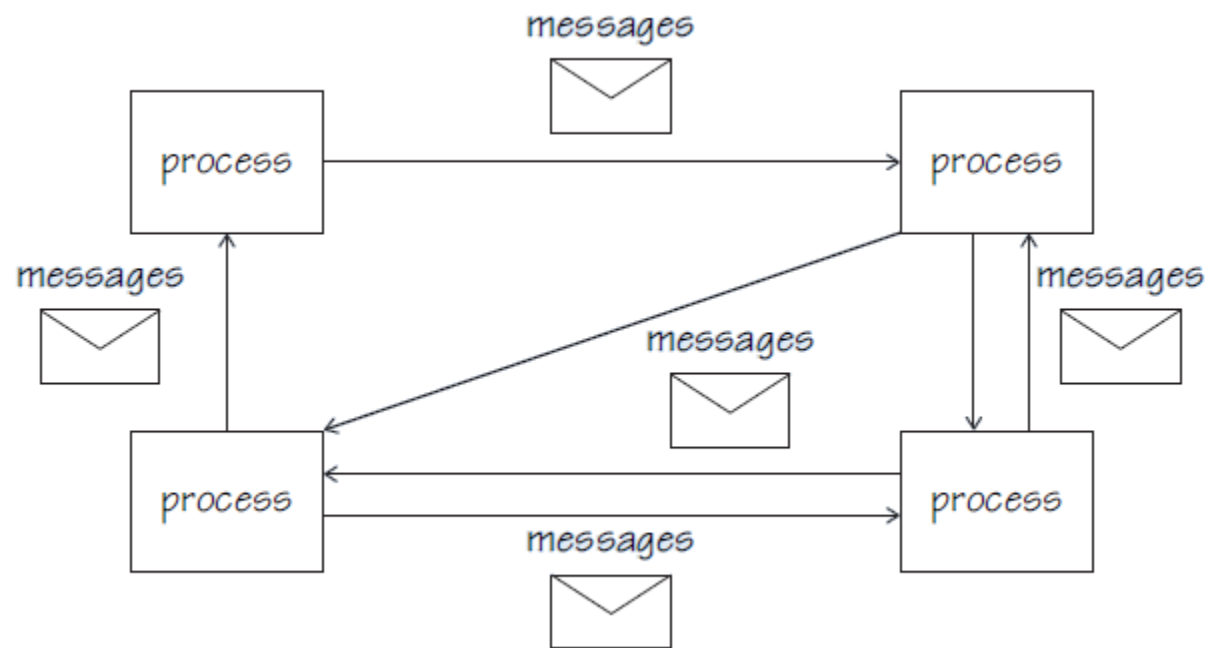
# SOFTWARE ARCHITECTURAL STYLES

## INDEPENDENT COMPONENTS ARCHITECTURE

Components execute concurrently and are decoupled as far as possible, but can communicate by messages that allow them to exchange data or coordinate their operations.

### Example:

Concurrent/Real-time Systems .. A set of components controlling different parts of a chemical processing plant, independently regulating the part each is responsible for, but sharing data and coordinating with one another by exchanging messages.

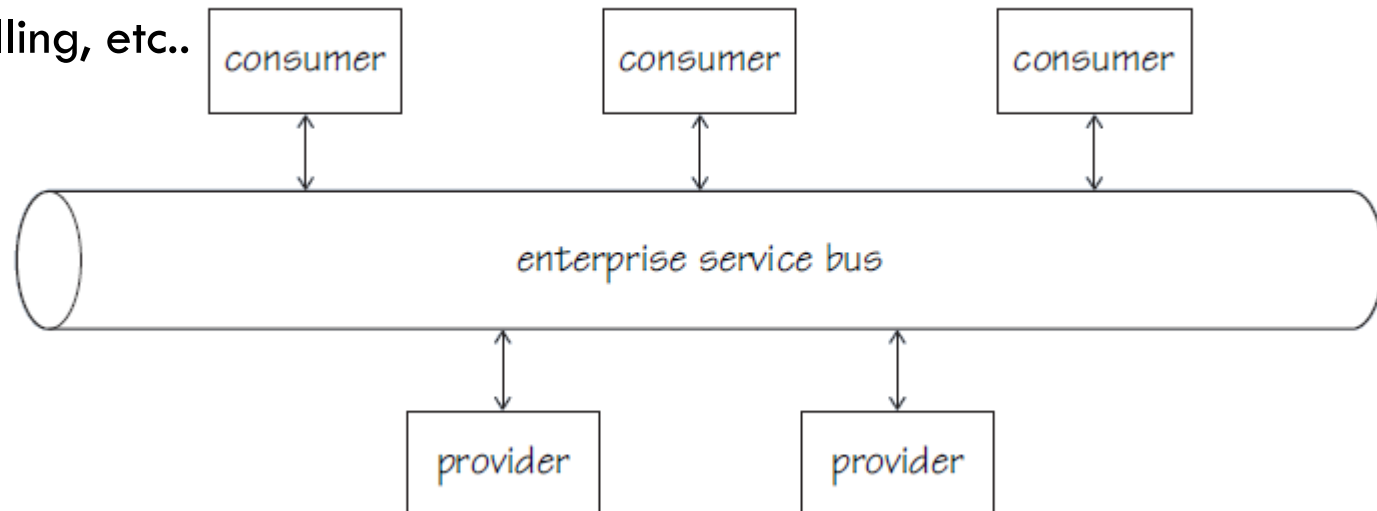


# SOFTWARE ARCHITECTURAL STYLES

## SERVICE-ORIENTED ARCHITECTURE

Two kinds of components; **consumers** & **providers**. A set of service providers makes services available to a set of service consumers. Consumers can combine services to carry out the business processes they require.

**Example:** Different divisions of an organization whose systems all use a common set of services such as payroll, personnel, customer records, management billing, etc..

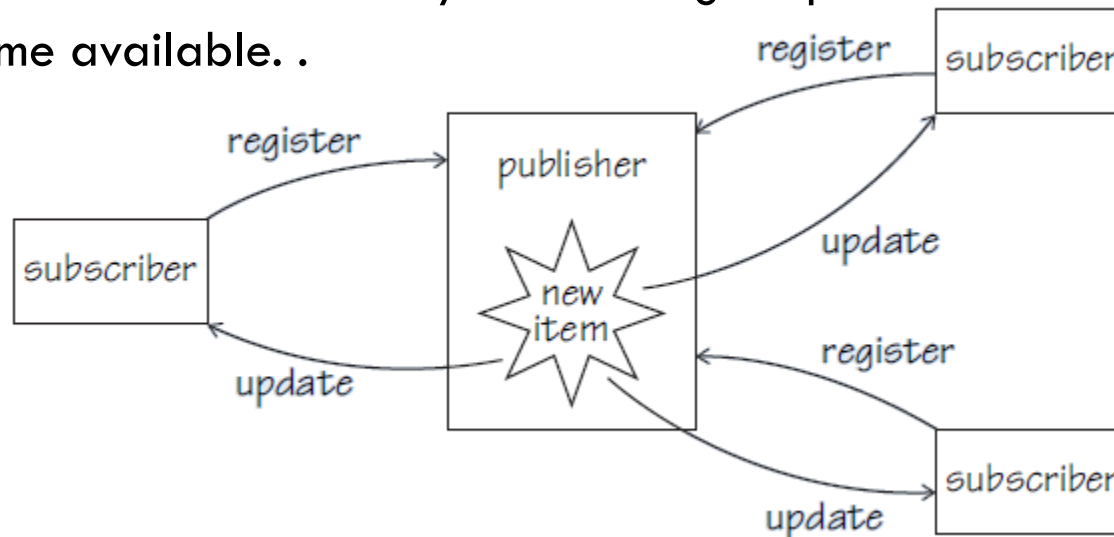


# SOFTWARE ARCHITECTURAL STYLES

## NOTIFICATION / IMPLICIT-INVOCATION / PUBLISH-SUBSCRIBE ARCHITECTURE

Two kinds of components; **observers** & **subjects**. Observers/subscribers can register themselves with a subject/publisher to be kept notified whenever some particular event happens at the subject's end.

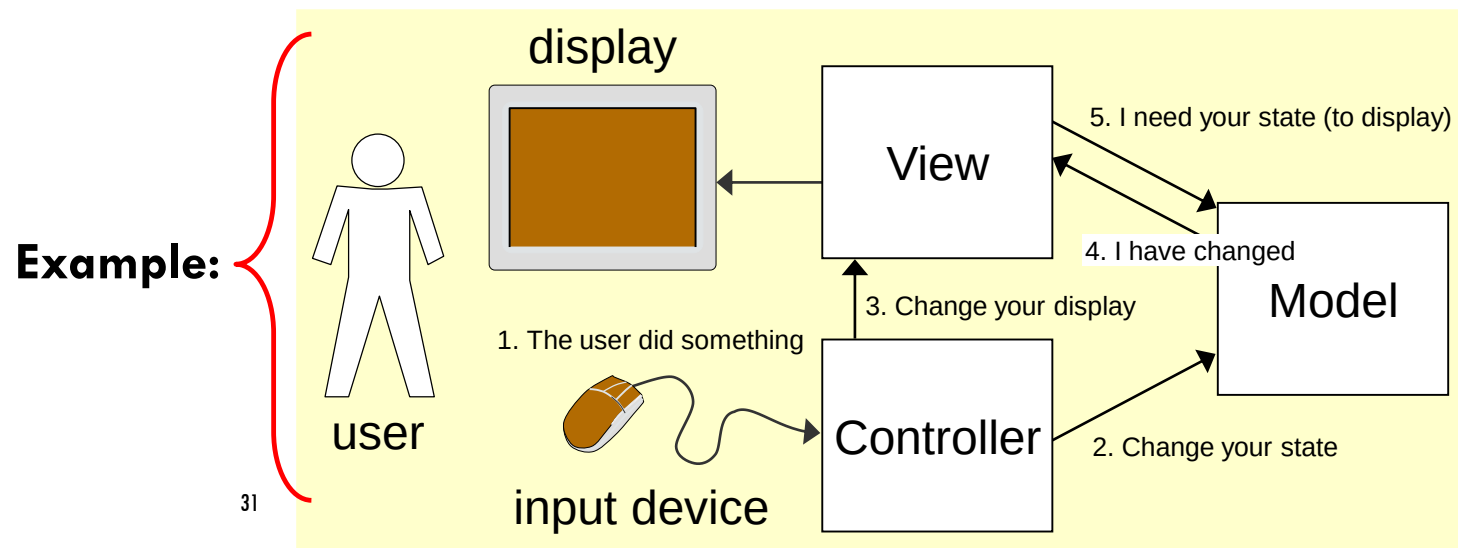
**Example:** RSS feed where anyone who signs up receives news updates as they become available. .



# SOFTWARE ARCHITECTURAL STYLES

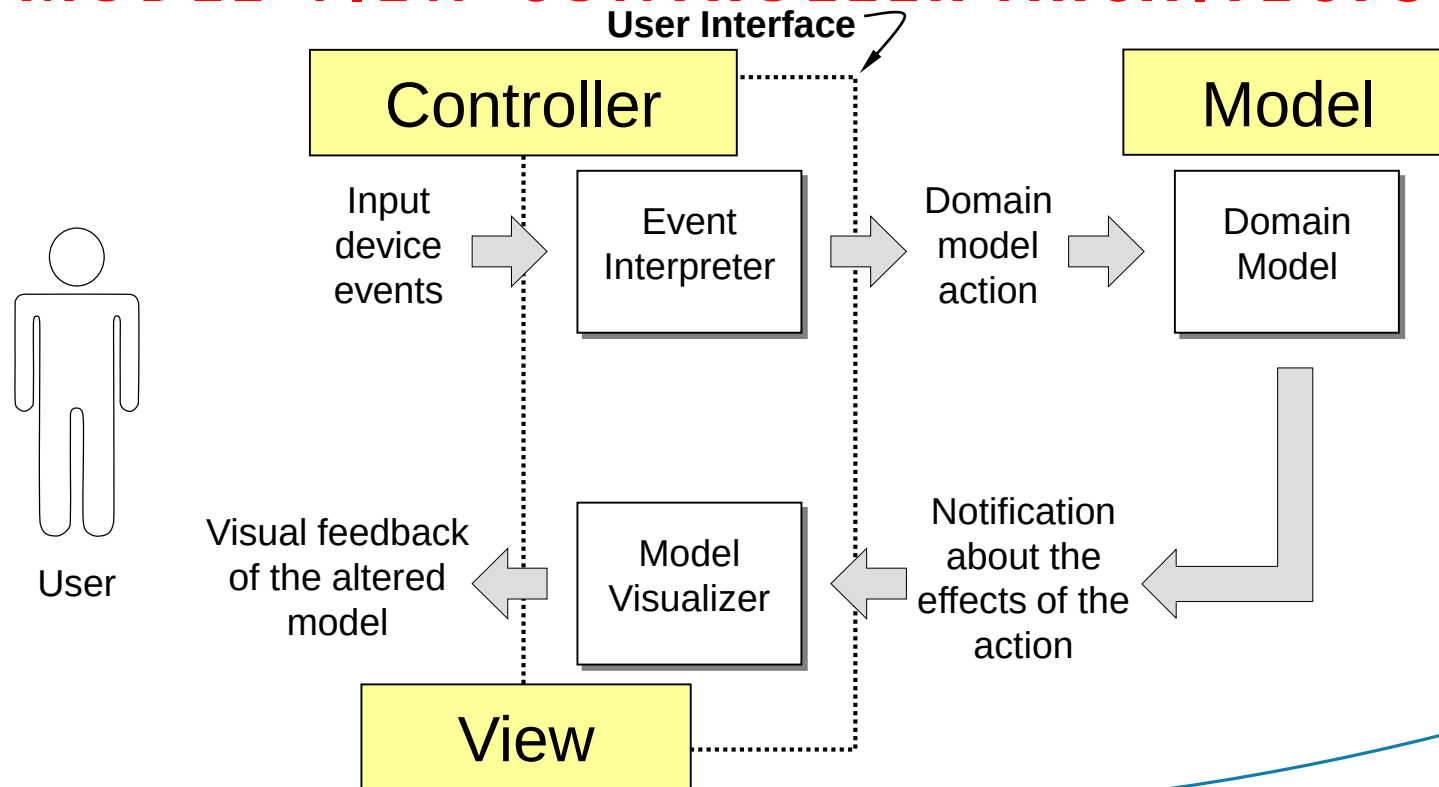
## MODEL-VIEW-CONTROLLER ARCHITECTURE

- **Model:** Holds all the data, state and application logic. Oblivious to the View and Controller. Provides API to retrieve state and send notifications of state changes to “observer”.
- **View:** Gives user a presentation of the Model. Gets data directly from the Model.
- **Controller:** Takes user input and figures out what it means to the Model.



# SOFTWARE ARCHITECTURAL STYLES

## MODEL-VIEW-CONTROLLER ARCHITECTURE

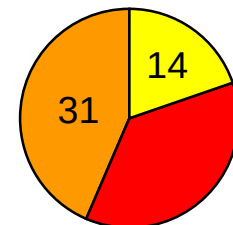


Model: array of numbers [ 14, 26, 31 ]

➔ Different Views for the same Model:



versus



# THANK YOU!

## Questions?

